

1.2 The Streaming Audio Preprocessing Pipeline

► LEARNING OBJECTIVE

Rather than treating the frontend as a recipe (frame $\rightarrow$ FFT $\rightarrow$ Mel $\rightarrow$ log), this section walks the physical packets that cross each boundary. We trace how an infinite air pressure wave is sampled, buffered, windowed, transformed, filtered, and compressed into deterministic feature frames, proving why on-chip memory layouts and pointer arithmetic must mirror the exact dimensions of the underlying acoustic tensors.

In Section 1.1 we established that acoustic physics forbids batching ($B > 1 = \text{latency}$). Yet, digital signal processors and neural acoustic models cannot ingest an infinite analog wave directly. To extract linguistic representations from continuous air pressure, we must construct a hardware-software datapath that slices, conditions, and transforms the streaming audio with absolute temporal determinism.

This section follows the dataflow across the six standard transformations implemented in the reference pipeline `chapter01/exp_01_streaming_audio_pipeline.py`: the raw waveform, the sliding ring buffer, the tapering window, the Short-Time Fourier Transform (STFT), the Mel filterbank, and logarithmic compression. These stages do not form an arbitrary software syllabus. Each layer arises because the physical stage immediately preceding it leaves an unresolved mechanical dilemma that prevents the next arithmetic operation from executing:

- Air pressure is continuous, whereas digital hardware stores discrete numbers; the physical wave must therefore be **sampled** at uniform clock intervals.
- The streaming audio has no end, whereas frequency analysis requires a stationary, finite block of numbers; the system must therefore maintain a fixed-size **buffer** that steps forward by a discrete hop.
- Slicing an ongoing wave creates abrupt discontinuities at the frame boundaries, which a Fourier analyzer misinterprets as spurious high-frequency harmonics; each frame must therefore be **windowed** (tapered toward zero) before transformation.
- Tapering the frame edges suppresses boundary energy where acoustic information resides; consecutive frames must therefore **overlap heavily**, triggering a full spectral analysis once every hop rather than once per window length.
- A windowed frame remains in the time domain, hiding resonance frequencies; the **STFT** evaluates the frame against N candidate probing frequencies to generate a spectral power representation.
- The resulting 257 linear frequency bins are too redundant for downstream acoustic models and follow a linear Hertz scale that ignores human hearing; they are pooled into 80 non-linear **Mel filterbank bands**.
- Energy values across spectral bands vary across multiple orders of magnitude, causing neural network additions to be dominated by the loudest band; each energy value is therefore compressed by taking its **logarithm**.

Step 1: The Waveform and Nyquist Sampling

Sound is recorded as a discrete time-series measuring fluctuations in atmospheric air pressure. The temporal spacing between measurements is dictated by the **sampling frequency** (f_s), defining the number of discrete samples captured per second.

The standard edge voice pipeline operates at $f_s = 16$ kHz. Every $T_s = 62.5 \mu\text{s}$, exactly one new sample emerges from the microphone analog-to-digital converter (ADC), and a 25 ms analysis window accumulates 400 samples. This parameter choice is rooted in the biomechanics of human vocal production:

- The vocal folds vibrate at a fundamental frequency (F_0) ranging from 85 Hz (deep male voice) to 255 Hz (high female voice).
- The resonant cavities of the vocal tract (pharynx, oral cavity, and nasal cavity) generate formant peaks (F_1, F_2, F_3) that define vowel identities, spanning from 300 Hz to 3,500 Hz.
- The highest acoustic frequencies in human speech occur during voiceless fricatives and plosives (such as /s/ and /sh/), which decay rapidly above 8 kHz.

According to the Nyquist-Shannon sampling theorem, perfect reconstruction of a band-limited signal requires a sampling frequency at least twice its highest frequency component ($f_s \geq 2B$). A sampling rate of 16 kHz places the Nyquist boundary at 8 kHz, encapsulating the entire human phonetic spectrum while rejecting ultrasonic ambient noise. Raising the rate to the CD standard (44.1 kHz) or studio standard (48 kHz) triples the on-chip buffer size, bus bandwidth, and arithmetic compute load without contributing any meaningful phonetic information to the acoustic model.

Step 2: The Sliding Ring Buffer

The front end observes a window of $L = 400$ samples every hop, advancing across the input stream in strides of $H = 160$ samples ($T_h = 10$ ms).

To understand how hardware resolves this overlap, consider two competing implementation hypotheses:

- **Hypothesis A (The Naive Software Shift):** The processor treats the frame buffer as a contiguous linear array. When 160 new samples arrive, the software executes a memory shift (`memmove`), copying the most recent 240 samples downward to the beginning of the buffer before appending the new 160 samples at the tail.
- **Hypothesis B (The Hardware Circular Pointer Buffer):** The memory addresses remain fixed in silicon storage. Instead of moving data elements, the architecture updates two circular address pointers that increment modulo 400.

The resolution demonstrates the efficiency of custom hardware over general-purpose memory buses: under Hypothesis A, the processor pays a recurring memory traffic tax on every frame (240 reads and 240 writes). Under Hypothesis B, the physical samples never move once written to Block RAM (BRAM). The write pointer advances by 160 addresses to overwrite the oldest data, while the read pointer streams 400 contiguous samples into the computation datapath starting from the updated boundary.

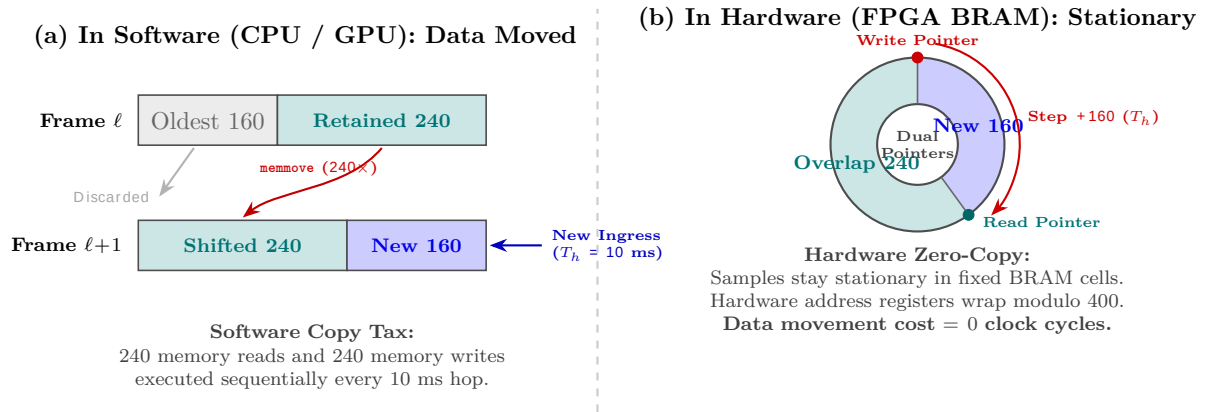


Figure 1.2a: The mechanical contrast between software memory shifting and hardware circular ring buffering for a 400-sample window with a 160-sample hop ($T_h = 10$ ms). Notice that while software moves 240 samples per frame via sequential reads and writes, the hardware architecture stores audio in fixed on-chip Block RAM cells and advances dual address registers modulo 400. This proves that dedicated FPGA storage eliminates data movement overhead entirely, reducing data transfer cost to zero clock cycles.

★ GROUNDBREAKING FINDING

On spatial FPGA architectures, data movement cost for frame overlap drops to zero:

$$\text{Data Movement Energy} \rightarrow 0$$

Inter-frame acoustic state does not live in software copy loops. It lives entirely in the address bits of circular BRAM pointer registers. This circular buffer forms the first persistent state module in the streaming voice pipeline.

Step 3: Windowing and the Boundary Discontinuity

A 400-sample analysis window represents a finite rectangular cut extracted from an ongoing analog sound wave. A rectangular truncation acts as a sharp step function: the signal starts instantaneously at sample 0 and cuts off abruptly at sample 399.

To a Fourier analyzer, an instantaneous vertical transition cannot be distinguished from high-frequency harmonics that never existed in the acoustic environment. If transformed directly, these boundary cliffs produce severe **spectral leakage**, smearing energy across adjacent frequency bins.

To prevent this distortion, the 400 samples stored in the ring buffer are multiplied element-wise by a symmetric **Hamming window** $w[n]$:

$$x_w[n] = x[n] \cdot w[n], \quad 0 \leq n < 400$$

The Hamming window tapers smoothly toward near-zero at both margins, ensuring that the waveform enters and exits the analysis frame without sharp step transitions.

► PHYSICAL INSIGHT

A tapering window is a boundary tax paid so that the Fourier analyzer does not encounter an artificial cliff. By forcing the boundary samples toward zero, the window suppresses spurious high-frequency harmonics at the cost of attenuating real edge energy. This attenuation causes spectral broadening, which is why consecutive frames must overlap heavily by 240 samples to ensure every acoustic event is analyzed near a window peak.

Step 4: The Short-Time Fourier Transform (STFT)

Computing the Discrete Fourier Transform (DFT) on consecutive windowed frames across time forms the Short-Time Fourier Transform (STFT).

To compute the DFT efficiently, the 400 windowed samples are extended to 512 samples by appending 112 zero values (**zero-padding**). Padding to a power of two ($N_{\text{fft}} = 512$) enables the use of radix-2 Fast Fourier Transform (FFT) pipelines.

Because the incoming audio signal consists entirely of real-valued numbers, the resulting complex spectrum possesses Hermitian symmetry: the upper half of the frequency bins is a redundant conjugate mirror of the lower half. The engine discards the redundant negative frequencies, retaining only the $N_{\text{fft}}/2 + 1 = 257$ non-redundant positive frequency bins. Taking the squared magnitude of these complex values yields the power spectrum:

$$P[k] = |X[k]|^2, \quad 0 \leq k \leq 256$$

Step 5: The Mel Filterbank

The 257 linear power spectrum bins provide uniform frequency resolution across the entire 0 to 8 kHz range ($8,000/256 = 31.25$ Hz per bin).

However, human auditory perception does not perceive frequency differences on a linear scale. The human cochlea resolves low-frequency tones with high precision while grouping high-frequency tones into broad perceptual bands. Furthermore, feeding 257 raw spectral values directly into a small neural network inflates the parameter count unnecessarily.

The pipeline compresses the 257 linear bins into $M = 80$ **Mel filterbank bands**. Each Mel filter is a triangular weighting function spanning a cluster of adjacent FFT bins. Filters are densely spaced at low frequencies and widen progressively at higher frequencies. The output energy $E[m]$ for the m -th Mel band is computed as the weighted sum of the linear power spectrum bins overlapping that filter:

$$E[m] = \sum_{k=0}^{256} W[m, k] \cdot P[k], \quad 0 \leq m < 80$$

where W is the precomputed 80×257 sparse triangular weighting matrix.

Step 6: Logarithmic Compression

Acoustic energy across the 80 Mel bands can vary by factors of millions between soft vocal murmurs and loud plosive bursts. If raw energy values were fed directly into a neural acoustic model, large amplitudes would saturate activation functions and dominate gradient updates.

Furthermore, human perception of loudness conforms to the Weber-Fechner law: the perceived increase in volume is proportional to the ratio of sound intensity rather than absolute linear differences.

The final preprocessing stage compresses each Mel energy band logarithmically:

$$S[m] = \ln \left(\max \left(E[m], 10^{-6} \right) \right), \quad 0 \leq m < 80$$

The floor threshold of 10^{-6} serves a critical arithmetic purpose: during periods of absolute silence where band energy approaches zero, the clamp prevents the logarithmic operator from evaluating to negative infinity ($-\infty$), guaranteeing numerical stability in fixed-point and floating-point hardware pipelines.

Tensor Dimensions and Memory Footprint

In silicon architectures, tensor dimensions are not abstract software dimensions; they dictate register bit-widths, Block RAM depth, and memory bus bandwidth. The complete data progression across a single 10 ms frame is summarized in Table 1:

Table 1: Streaming audio tensor dimensions and FP32 memory sizes per 10 ms frame.

Processing Layer	Data Produced per Frame	FP32 Footprint (Float 32-bit)
Incoming Waveform	160 new samples	640 B loaded every 10 ms
Sliding Ring Buffer	400 resident samples	1,600 B, persistent on-chip state
Windowed Frame	400 windowed samples	1,600 B (buffer $\times$ Hamming window)
FFT Input Buffer	512 samples	2,048 B (400 real samples + 112 zero-pad)
Power Spectrum	257 spectral bins	1,028 B (257 real energy values)
Mel Filterbank Bands	80 filterbank values	320 B (80×257 matrix multiply)
Log-Mel Feature Frame	80 log-energy features	320 B (direct input to acoustic model)

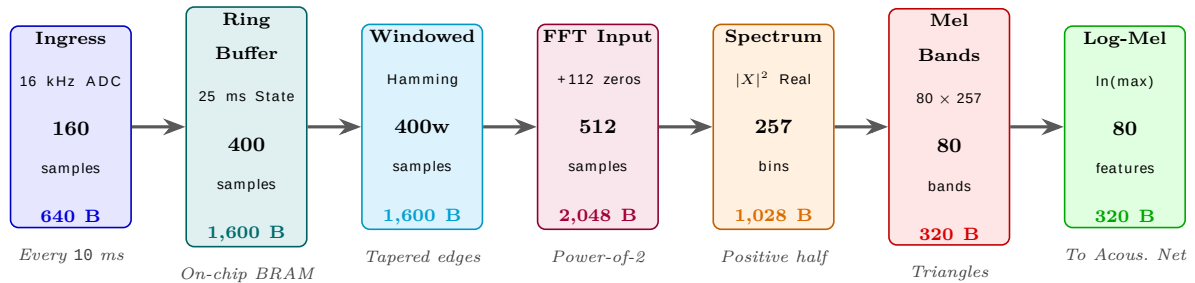


Figure 1.2b: Tensor dimensions and memory footprint along the streaming audio datapath for one 10 ms analysis frame. Notice how the data representation expands from 160 samples (640 B) to 512 zero-padded points (2,048 B) for spectral computation, then compresses down to 80 compact log-energy values (320 B) for neural network inference. This proves that hardware buffer sizing and bus widths are directly dictated by the mathematical tensor shapes of the DSP pipeline.

Misconception Buster: Three Traps in Streaming Voice Preprocessing

The following cognitive traps frequently mislead developers transitioning from offline machine learning or general-purpose computing to edge FPGA implementation:

MISCONCEPTION BUSTER: Trap A — Higher Sampling Rate (48 kHz) Improves AI Accuracy

Root Cause: Hi-fi audio intuition conflates higher sampling rates (48 kHz or 96 kHz) with crisper musical reproduction, leading developers to assume higher rates increase acoustic model accuracy.

Scientific Reality: Human speech formants (F_1, F_2, F_3) reside between 300 Hz and 3,500 Hz, while voiceless fricatives decay before 8 kHz. Raising f_s to 48 kHz captures only ambient ultrasonic hiss while tripling on-chip BRAM allocation, bus bandwidth, and FFT arithmetic complexity, offering zero phonetic advantage to the acoustic model.

MISCONCEPTION BUSTER: Trap B — The Sliding Frame Buffer Is Merely a Software Memory Copy

Root Cause: In standard Python or C implementations, stepping the analysis frame is executed using memory shift routines (`memmove`), requiring 240 read and 240 write cycles on every hop.

Scientific Reality: On spatial FPGA hardware, a circular buffer consumes zero copy clock cycles. By implementing two hardware counter registers that cycle around on-chip Block RAM, audio samples remain stationary while addresses wrap modulo 400. The sequential $O(L - H)$ memory transfer overhead disappears entirely.

MISCONCEPTION BUSTER: Trap C — Rectangular Window Slicing Preserves Unaltered Audio Samples

Root Cause: A flat rectangular cut preserves the exact numerical values of raw audio samples without multiplying by scaling coefficients, creating the illusion of perfect signal fidelity.

Scientific Reality: Sharp step discontinuities at frame boundaries generate spurious high-frequency harmonics absent from the recording room. To a Fourier transform, sudden edges resemble infinite-frequency impulses that cause severe spectral leakage across adjacent bins. Applying a smooth tapering window (Hamming) is physically mandatory to eliminate boundary artifacts.

The Unresolved Dilemma: The Frequency Blur

Through the six stages of this pipeline, we have successfully conditioned raw air pressure waves into stationary 80-dimensional feature vectors emitted once every 10 ms.

Yet, this transformation reveals a fundamental physical tension: when we squeezed the analysis window down to 25 ms to preserve temporal stationarity, what happened to our ability to distinguish two closely spaced resonance frequencies? Why does an FFT bin spanning 31.25 Hz blur adjacent harmonics, and how does the Mel filterbank balance this trade-off between time resolution and frequency resolution?

This mathematical dilemma brings us directly to the formal derivation of the STFT and Mel filterbank in Section 1.3.